Project – Mini Compiler for Arithmetic Expressions (CSF401) , VIIth SEM , 2025-26

Student Name: Harshita Hoiyani SAP ID: 1000018489 Section: D

Project Name: ***DP1-6 Mini Compiler for Arithmetic Expressions***

For this project, you will solve problems based on what you have learned in **this course**.

Instructions ***// These are sample instructions, you may keep relevant ones and remove/add rest as required.***

- The project must be developed in C/Lex.
- Mention names and IDs of all group members on the top of this document.
- Some objectives are given under project title. Mention the project title you have chosen on the top of this page. In each question, objective to be achieved, expected inputs and expected outputs are mentioned.
- **Project submitted after due date will not be evaluated and a score of zero will be awarded.**
- Upload a **pdf version** of the document.
- The group size should not be more than 3, and the group must be formed from the same section. Inter section groups will not be accepted.
- Plagiarism may lead to award of zero marks.
- If you have doubts about any project, please contact the course instructor.

Due Date: **23:59 PM, November, 03, 2025.**

Submitting this Project

You will submit (upload) this project in LMS. Email/paper submissions will not be accepted.

- Write your answer (codes and snapshots of input and output) after the question in this document.
- Name this document as Project_CD_ODD2025_SAP1_SAP2_SAP3.doc in case group members IDs are SAP1, SAP2 and SAP3 respectively, acronym of course is CD, Sem is ODD and year is 2025.

Project – Mini Compiler for Arithmetic Expressions (CSF401) , VIIth SEM , 2025-26

Grading Scheme: This project has 10 marks. Marks distribution within this project is as follows //(Faculty may change this text and distribution as deemed fit):

- Appropriate GUI for input and output as mentioned in the question: X marks,
- Expected input and Output as mentioned in the question: Y marks,
- Well documented code/Presentation = Z marks.

// Where X+Y+Z = 10

// A Sample Project is mentioned , showing (a) objective, (b) expected inputs, // and (c) expected outputs. (a) (b) and (c) must be mentioned very clearly // using correct English, and there should be no ambiguity.

Project-DP1-6: Mini Compiler for Arithmetic Expressions

Objective: - To design a mini - compiler that integrates lexical analysis, syntax analysis, and intermediate code generation for arithmetic expressions using c platform. The goal is to understand how a compiler processes an expression by tokenizing it, validating its structure using grammar rules, and generating corresponding **Three Address Code (TAC)**.

Inputs: The program accepts a **single-line arithmetic expression** from the user that may include:

- Integers (e.g., `3`, `45`, `100`)
- Arithmetic operators: `+`, `-`, `*`, `/`
- Parentheses for grouping: `(` and `)`
- Optional spaces between symbols

Example Input: **3 + 4 * (2 - 1)**

10 / (5 - 3)

-5 + 2 * 3

Project – Mini Compiler for Arithmetic Expressions (CSF401) , VIIth SEM , 2025-26

Outputs: The program generates the **intermediate code** (Three Address Code) for the given expression, showing step-by-step evaluation using temporary variables (`t1`, `t2`, …).
It also prints the final temporary variable where the result is stored.

Example Output:

```
t1 = 3
t2 = 4
t3 = 2
t4 = 1
t5 = t3 - t4
t6 = t2 * t5
t7 = t1 + t6
RESULT in t7
```

Language/Platform Used:

- **Language:** C
- **Compiler:** GCC (MinGW / GCC for Linux/macOS)
- **IDE:** Visual Studio Code

Algorithm/Approach:

The compiler is implemented using **Recursive Descent Parsing** for the following grammar:

Expr → Term { ('+' | '-') Term }
Term → Factor { ('*' | '/') Factor }
Factor → NUMBER | '(' Expr ')' | '-' Factor

1. **Lexical Analysis:** Tokenizes the input into numbers, operators, and parentheses.
2. **Syntax Analysis:** Validates structure and operator precedence.
3. **Intermediate Code Generation:** Produces TAC showing order of evaluation.

Project – Mini Compiler for Arithmetic Expressions (CSF401) , VIIth SEM , 2025-26

Snapshot Of Code Implementation:

```
/*
Mini-compiler for arithmetic expressions
Language: C (single-file)

What this does:
- Performs lexical analysis (tokenizer) for arithmetic expressions containing
integers, parentheses, and operators + - * /
- Performs syntax analysis using a recursive-descent parser for the grammar:

    Expr   -> Term { ('+' | '-') Term }
    Term   -> Factor { ('*' | '/') Factor }
    Factor -> NUMBER | '(' Expr ')' | '-' Factor

- Generates intermediate code in the form of three-address code (TAC).
- Prints the TAC to stdout and shows the final temporary holding the expression
result.

How to compile:
    gcc -o mini_compiler Mini-compiler_for_arithmetic_expressions.c

How to run:
    ./mini_compiler
    Then enter one-line arithmetic expressions, e.g.:
    3 + 4 * (2 - 1)

Sample output:
    t1 = 3
    t2 = 4
    t3 = 2
    t4 = 1
    t5 = t3 - t4
    t6 = t2 * t5
    t7 = t1 + t6
    RESULT in t7

Notes / next steps you can add:
- Support variables and assignments (e.g. a = 3 + b)
- Support unary plus, floats, or exponentiation
- Build a symbol table and generate code in quadruple format
```

Project – Mini Compiler for Arithmetic Expressions (CSF401) , VIIth SEM , 2025-26

```
- Add simple optimization: constant folding, dead-code elimination
- Output to a file or generate assembly-like output

-- END OF HEADER --
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* ----- Lexer------ */

enum TokenType { T_END, T_NUM, T_PLUS, T_MINUS, T_MUL, T_DIV, T_LPAREN, T_RPAREN,
T_INVALID };

typedef struct {
    enum TokenType type;
    char lexeme[64];
    int value; // for numbers
} Token;

const char *input;
int pos;
Token curToken;

void skip_spaces() {
    while (input[pos] && isspace((unsigned char)input[pos])) pos++;
}

void nextToken() {
    skip_spaces();
    char c = input[pos];
    if (c == '\0' || c == '\n') {
        curToken.type = T_END;
        return;
    }
    if (isdigit((unsigned char)c)) {
        int val = 0;
        int i = 0;
        while (isdigit((unsigned char)input[pos])) {
```

Project – Mini Compiler for Arithmetic Expressions (CSF401) , VIIth SEM , 2025-26

```
            if (i < 60) curToken.lexeme[i++] = input[pos];
            val = val * 10 + (input[pos++] - '0');
        }
        curToken.lexeme[i] = '\0';
        curToken.type = T_NUM;
        curToken.value = val;
        return;
    }
    switch (c) {
        case '+': curToken.type = T_PLUS; curToken.lexeme[0] = '+'; curToken.lexeme[1]
= '\0'; pos++; return;
        case '-': curToken.type = T_MINUS; curToken.lexeme[0] = '-';
curToken.lexeme[1] = '\0'; pos++; return;
        case '*': curToken.type = T_MUL; curToken.lexeme[0] = '*'; curToken.lexeme[1]
= '\0'; pos++; return;
        case '/': curToken.type = T_DIV; curToken.lexeme[0] = '/'; curToken.lexeme[1]
= '\0'; pos++; return;
        case '(': curToken.type = T_LPAREN; curToken.lexeme[0] = '(';
curToken.lexeme[1] = '\0'; pos++; return;
        case ')': curToken.type = T_RPAREN; curToken.lexeme[0] = ')';
curToken.lexeme[1] = '\0'; pos++; return;
        default:
            curToken.type = T_INVALID;
            curToken.lexeme[0] = input[pos++]; curToken.lexeme[1] = '\0';
            return;
    }
}

/* ----- Intermediate code representation (simple)-------*/

typedef struct {
    char res[32];
    char arg1[32];
    char op[8];
    char arg2[32];
} Instr;

#define MAX_INSTR 2000
Instr instrs[MAX_INSTR];
int instr_count = 0;
int temp_count = 0;
```

Project – Mini Compiler for Arithmetic Expressions (CSF401) , VIIth SEM , 2025-26

```
char *new_temp() {
    char *s = (char *)malloc(16);
    snprintf(s, 16, "t%d", ++temp_count);
    return s;
}

void emit(const char *res, const char *arg1, const char *op, const char *arg2) {
    if (instr_count >= MAX_INSTR) {
        fprintf(stderr, "instruction buffer overflow\n");
        exit(1);
    }
    strncpy(instrs[instr_count].res, res, sizeof(instrs[instr_count].res)-1);
    instrs[instr_count].res[sizeof(instrs[instr_count].res)-1] = '\0';
    strncpy(instrs[instr_count].arg1, arg1, sizeof(instrs[instr_count].arg1)-1);
    instrs[instr_count].arg1[sizeof(instrs[instr_count].arg1)-1] = '\0';
    strncpy(instrs[instr_count].op, op, sizeof(instrs[instr_count].op)-1);
    instrs[instr_count].op[sizeof(instrs[instr_count].op)-1] = '\0';
    strncpy(instrs[instr_count].arg2, arg2, sizeof(instrs[instr_count].arg2)-1);
    instrs[instr_count].arg2[sizeof(instrs[instr_count].arg2)-1] = '\0';
    instr_count++;
}

/* ----- Parser (recursive descent) returns the name of the place (temp or literal)
holding value------ */

char *parse_expr();
char *parse_term();
char *parse_factor();

void expect(enum TokenType t) {
    if (curToken.type == t) nextToken();
    else {
        fprintf(stderr, "Syntax error: expected token %d but found '%s'\n", t,
curToken.lexeme);
        exit(1);
    }
}

char *num_to_place(int value) {
    // Create a temp to hold literal (we could also directly use the constant lexeme)
```

Project – Mini Compiler for Arithmetic Expressions (CSF401) , VIIth SEM , 2025-26

```
    char *place = new_temp();
    char arg[32];
    snprintf(arg, sizeof(arg), "%d", value);
    emit(place, arg, "=", "");
    return place;
}

char *parse_factor() {
    if (curToken.type == T_NUM) {
        int v = curToken.value;
        nextToken();
        return num_to_place(v);
    } else if (curToken.type == T_LPAREN) {
        nextToken();
        char *p = parse_expr();
        if (curToken.type != T_RPAREN) {
            fprintf(stderr, "Syntax error: missing ')'\n"); exit(1);
        }
        nextToken();
        return p;
    } else if (curToken.type == T_MINUS) {
        // unary minus
        nextToken();
        char *f = parse_factor();
        char *t = new_temp();
        emit(t, "0", "-", f);
        return t;
    } else {
        fprintf(stderr, "Syntax error: unexpected token '%s' in factor\n",
curToken.lexeme);
        exit(1);
    }
}

char *parse_term() {
    char *left = parse_factor();
    while (curToken.type == T_MUL || curToken.type == T_DIV) {
        enum TokenType op = curToken.type;
        nextToken();
        char *right = parse_factor();
        char *t = new_temp();
```

Project – Mini Compiler for Arithmetic Expressions (CSF401) , VIIth SEM , 2025-26

```
        if (op == T_MUL) emit(t, left, "*", right);
        else emit(t, left, "/", right);
        left = t;
    }
    return left;
}

char *parse_expr() {
    char *left = parse_term();
    while (curToken.type == T_PLUS || curToken.type == T_MINUS) {
        enum TokenType op = curToken.type;
        nextToken();
        char *right = parse_term();
        char *t = new_temp();
        if (op == T_PLUS) emit(t, left, "+", right);
        else emit(t, left, "-", right);
        left = t;
    }
    return left;
}

int main() {
    char buffer[1024];
    printf("Enter an arithmetic expression (single line).\n");
    if (!fgets(buffer, sizeof(buffer), stdin)) return 0;
    input = buffer;
    pos = 0;
    instr_count = 0;
    temp_count = 0;
    nextToken();
    char *result_place = parse_expr();
    if (curToken.type != T_END && curToken.type != T_RPAREN) {
        fprintf(stderr, "Syntax error: unexpected token after expression '%s'\n",
curToken.lexeme);
        return 1;
    }

    // Print TAC
    for (int i = 0; i < instr_count; ++i) {
        Instr *ins = &instrs[i];
        if (strcmp(ins->op, "=") == 0) {
```

Project – Mini Compiler for Arithmetic Expressions (CSF401) , VIIth SEM , 2025-26

```
            // assignment of literal
            if (strlen(ins->arg2) == 0)
                printf("%s = %s\n", ins->res, ins->arg1);
            else
                printf("%s = %s %s %s\n", ins->res, ins->arg1, ins->op, ins->arg2);
        } else {
            if (strlen(ins->arg2) == 0)
                printf("%s = %s %s\n", ins->res, ins->arg1, ins->op);
            else
                printf("%s = %s %s %s\n", ins->res, ins->arg1, ins->op, ins->arg2);
        }
    }
    printf("RESULT in %s\n", result_place);

    return 0;
}
```

Snapshot Of Code Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

/Users/harshitahoiyani/.zshrc:export:1: not valid in this context: /Users/harshitahoiyani/.zshrc
harshitahoiyani@Harshitas-MacBook-Air ~ % gcc -o mini_compiler mini_compiler.c
ld: warning: reducing alignment of section __DATA,__common from 0x8000 to 0x4000 because it exceeds segment maximum alignment
harshitahoiyani@Harshitas-MacBook-Air ~ % ./mini_compiler
Enter an arithmetic expression (single line).
3+4*(2-1)
t1 = 3
t2 = 4
t3 = 2
t4 = 1
t5 = t3 - t4
t6 = t2 * t5
t7 = t1 + t6
RESULT in t7
harshitahoiyani@Harshitas-MacBook-Air ~ %
```